

AI FitTrack API – Project Documentation

Project Title: AI FitTrack API

Team Leader:

M. Meera

Email: meeracp4567@gmail.com

Team Members:

S. Sridevi

Email: sridevisrinivasancpm@gmail.com

S. Suriyakala

Email: suthasuriyakala@gmail.com

C. Kalpana

Email: ckalpu435@gmail.com

S. Kersiyal

Email: kersikersi443@gmail.com

AI FitTrack API is a RESTful backend application designed to provide fitness tracking, user management, workout and activity recording, and AI-powered fitness insights. The system uses Node.js and Express.js for backend API development and MongoDB with Mongoose for persistent storage. It can provide personalized fitness summaries and practical recommendations based on user activity data. The project follows a modular backend architecture using routes, controllers, models, middleware, and services.

Objectives

- Provide RESTful APIs for fitness and activity tracking.
- Implement secure user registration, login and profile retrieval.
- Protect private APIs using JSON Web Tokens (JWT).
- Store user accounts, fitness activities and workout information in MongoDB.
- Allow users to add, view, update and delete fitness activity records.
- Generate concise AI-powered fitness summaries and practical recommendations.

- Maintain application functionality through deterministic fallback logic when external AI services are unavailable.
- Provide a Postman collection for API testing.

System Overview

The application follows an MVC-oriented layered architecture. Incoming HTTP requests are received by Express routes, validated and processed by controllers, and then passed to models or service modules as required. Authentication middleware verifies JWT tokens before private operations. Fitness data is stored in MongoDB, while AI-related requests are handled through a dedicated AI service module.

Technologies Used

- Node.js – JavaScript runtime for the backend.
- Express.js – Web framework for REST API routing and middleware.
- MongoDB – NoSQL database for users and fitness activity data.
- Mongoose – ODM used to define schemas and communicate with MongoDB.
- JWT / jsonwebtoken – Token-based authentication and authorization.
- bcryptjs – Password hashing and password comparison.
- Google Gemini – AI generation of fitness summaries and recommendations.
- dotenv – Loads environment variables from .env.
- CORS – Cross-origin resource sharing middleware.
- Postman – API testing using the included collection.

Project Structure

Backend/

```

├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── fitnessController.js
│   │   └── aiController.js
│   ├── middleware/
│   │   └── authMiddleware.js
│   ├── models/
│   │   ├── User.js
│   │   └── FitnessActivity.js
│   └── routes/

```

```
| | |— authRoutes.js
| | |— fitnessRoutes.js
| | |— aiRoutes.js
| |— services/
| | |— aiService.js
| |— app.js
| |— server.js
|— postman_collection.json
|— package.json
|— package-lock.json
|— .env
|— README.md
```

Explanation of Folders

config

Contains configuration modules, including the MongoDB connection.

controllers

Contains functions that validate requests, invoke services/models, and return API responses.

middleware

Contains authentication middleware used to protect private routes.

models

Defines MongoDB schemas using Mongoose.

routes

Maps HTTP methods and URL paths to controller functions.

services

Handles AI integration and fallback logic.

app.js

Configures Express, middleware, routes and error handling.

server.js

Main entry point that establishes the database connection and starts the server.

Features

- User registration and login
- JWT-based authentication
- User profile retrieval
- Add fitness activity
- View fitness activities
- Update fitness activity
- Delete fitness activity
- Fitness progress tracking
- AI fitness summary
- AI fitness recommendation
- REST API based architecture
- MongoDB database integration
- Deterministic fallback AI logic

API Operations

Register User

Creates a new user account and returns a JWT.

Login User

Authenticates a user and returns a JWT.

Get Profile

Returns the authenticated user's profile.

Create Fitness Activity

Adds a workout or fitness activity for the authenticated user.

Read Fitness Activities

Retrieves stored fitness activity records.

Update Fitness Activity

Modifies an existing fitness activity owned by the user.

Delete Fitness Activity

Removes a fitness activity owned by the user.

Fitness Summary

Generates a summary based on recorded fitness activity.

Fitness Recommendation

Generates practical recommendations based on activity data.

API Endpoints

- POST /api/auth/register – Create a new user account.
- POST /api/auth/login – Authenticate a user.
- GET /api/auth/profile – Get the authenticated user's profile.
- POST /api/fitness – Add a fitness activity.
- GET /api/fitness – List fitness activities.
- PUT /api/fitness/:id – Update a fitness activity.
- DELETE /api/fitness/:id – Delete a fitness activity.
- POST /api/ai/fitness-summary – Generate an AI fitness summary.
- POST /api/ai/fitness-recommendation – Generate an AI fitness recommendation.

Database Design

User Collection

- name – Required string.
- email – Required, unique, lowercase and validated.
- password – Required string with a minimum of 6 characters and stored as a bcrypt hash.
- createdAt / updatedAt – Automatically managed by Mongoose timestamps.

FitnessActivity Collection

- user – Required ObjectId reference to User.
- activityType – Type of workout or fitness activity.
- duration – Duration of the activity in minutes.
- calories – Estimated calories burned.
- steps – Number of steps, when applicable.
- date – Date of the recorded activity.
- createdAt / updatedAt – Automatically managed by Mongoose timestamps.

Security Features

- Passwords are hashed with bcryptjs before database storage.
- JWT tokens are used for authenticated API access.
- Protected routes verify token validity before processing user-specific data.
- Fitness activity operations verify that the authenticated user owns the requested record.
- User profile retrieval excludes the password field.
- Environment variables are used for database credentials, JWT secrets and external AI API keys.

AI Fitness Integration

The AI service receives relevant fitness activity information and generates short, natural and practical fitness summaries and recommendations. The AI layer can interpret activity patterns such as workout duration, activity type, steps and calories and present the information in an easy-to-understand form.

AI Fitness Summary

Generates a concise summary of the user's recorded fitness activities. The summary can describe recent activity, workout consistency and basic progress based on the available data.

AI Fitness Recommendation

Generates practical recommendations based on the user's recorded activity data. Recommendations can focus on maintaining regular activity, improving consistency and setting realistic fitness goals.

Fallback and Resilience

- AI endpoints can generate local rule-based summaries when the external AI service is unavailable.
- Deterministic fallback logic supports local development and API testing.
- The application can continue basic fitness tracking even when AI credentials are not configured.

How to Run the Project

1. Install Node.js 18 or later and npm.
2. Open a terminal in the Backend project directory.
3. Run npm install to install dependencies.
4. Configure the required environment variables in the .env file.
5. Start development mode with npm run dev.
6. Use npm start for normal server execution.

API Testing with Postman

The project can use a Postman collection with sample requests for authentication, fitness activities and AI insights. The collection can use variables for base_url, token and activity_id. Private requests use Bearer token authentication.

7. Register a user.
8. Log in and obtain the JWT token.
9. Request the user profile.
10. Add a fitness activity and record its ID.
11. List, update and delete fitness activities.
12. Request an AI fitness summary.
13. Request an AI fitness recommendation.

Error Handling

Controllers use try/catch blocks and return structured JSON error messages. Express also defines a 404 handler for unknown routes and a centralized error handler for unhandled exceptions. Database startup failures prevent the server from running.

- 400 – Invalid or incomplete request data.
- 401 – Authentication/authorization failure.
- 404 – Requested activity or resource was not found.
- 500 – Unexpected server-side failure.

Non-Functional Characteristics

- Security – JWT authentication and bcrypt password hashing.
- Maintainability – Separation into routes, controllers, services, models and middleware.
- Reliability – AI fallback mechanisms.
- Testability – Postman collection and deterministic fallback logic.
- Scalability foundation – Stateless JWT authentication and modular service architecture.
- Data integrity – Mongoose validation and user-specific activity handling.

Current Limitations and Possible Enhancements

- Add request-rate limiting and stronger API abuse protection.
- Use stricter production validation and sanitization for all user inputs.
- Add structured logging and monitoring.
- Add automated unit and integration tests.
- Move production secrets to a secure secrets-management system.
- Add refresh-token or session-management strategy if required by the frontend.
- Add pagination if fitness activity collections become large.
- Add richer AI personalization using user goals and activity history.
- Add dashboards and graphical fitness progress visualization.
- Deploy the backend behind HTTPS with production-grade configuration.

Advantages

- Modular and maintainable backend architecture.
- Secure authentication and user-specific data handling.
- Efficient fitness data management using MongoDB.
- RESTful API design.
- AI-assisted interpretation of fitness activity.
- Fallback logic for development and testing.
- Easy integration with frontend applications.
- Supports personalized fitness tracking and recommendations.

Conclusion

AI FitTrack API provides a modular backend for a fitness-focused application with authentication, persistent fitness activity records, AI-assisted summaries and practical recommendations. Its MVC-style organization, JWT security, MongoDB data layer, AI service integration and fallback mechanisms make the implementation suitable for frontend integration and academic project demonstration. The API structure also supports straightforward testing through Postman.

References

- [Node.js Documentation](#)
- [Express.js Documentation](#)
- [MongoDB Documentation](#)
- [Mongoose Documentation](#)
- [JSON Web Token Documentation](#)
- [Google Gemini API Documentation](#)
- [Postman Documentation](#)